ODSI
Office of Data Science and Informatics

SNWG · HLS

Finding Efficiencies in HLS Production

What six years of operating HLS have taught us about cost and risk, and what other solutions can take from it

Sean Harkins · Development Seed

ORIENTATION

HLS at a glance

2020
Global v2.0 production running continuously since

15,000
Landsat and Sentinel-2 granules processed a day

~$450k
AWS spend in 2025 (about $418k after credits)

Harmonized Landsat and Sentinel-2 turns two missions into **one consistent 30 m surface reflectance record**, distributed by LP DAAC.

HLS is also an input. **Several other SNWG solutions build on it**, so our cost and latency become part of theirs. More on that at the end.

BACKGROUND

Why now

6 Six years of operations

- Cloud architecture patterns have changed a lot since 2020
- At scale, the hard problems are failures, accounting and cost, not the happy path
- Our riskiest code was the science code we did not control

A new requirement

- The HLS low-latency solution needs new auxiliary data and faster turnaround
- Patching the old system for it would add to the debt
- So we used it as the opportunity to **re-architect**

OUR GOALS

What we set out to improve

0

Low latency

The trigger: meet the new low-latency targets.

1

Science-code risk

Own and test the core algorithm code we depend on.

2

Cost at its floor

Pay for processing, not for waiting.

3

Reprocessing at scale

No bottleneck that blocks a 30M-granule campaign.

4

Observability

Answer "what happened to every granule?" quickly.

Where the 2025 money went



Share of about $450k gross AWS spend, 2025. The LaSRC work targets compute; the orchestration work targets part of "other" and unlocks cheaper reprocessing.

PART ONE

Modernizing LaSRC

Turning the science code at the heart of HLS into production code we can own, test and improve

THE PROBLEM

Science code is not production code

Science code optimizes for

- Answering a research question correctly
- Rapid iteration on the algorithm
- Being run by the person who wrote it

A production system requires

- Reproducibility across millions of granules
- Change that is isolated, reviewable and reversible
- Operation by people who did not write it, for a decade

For the USGS ESPA LaSRC code HLS has used since 2019, that gap looked like: **one maintainer** · **a deprecated build chain** · **no public repository or tagged releases** · **separate, non-interoperable VIIRS and MODIS auxiliary paths** · **HLS code coupled to LaSRC internals**

THE TRIGGER

The 5–6 month problem

5–6 months of one engineer

Our estimate for porting and testing LaSRC. For a small team running production while piloting other SNWG solutions, it never made the top of the list.

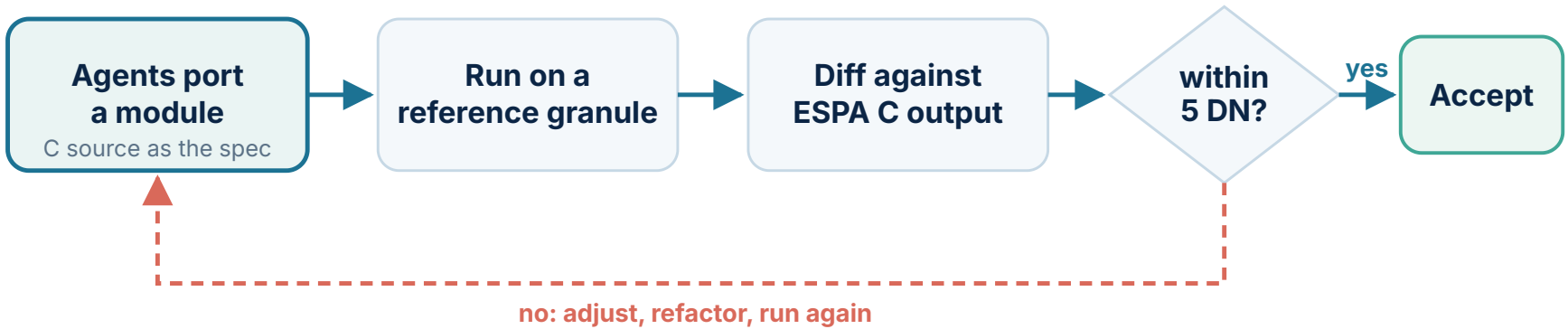
SEAN: CONFIRM

Days or weeks? State how long the agent-assisted port actually took.

1. **Low latency needs a new auxiliary source.** NOAA GDAS, instead of the slower VIIRS and MODIS products.
2. **ESPA cannot mix auxiliary sources.** Adding GDAS meant large, hard-to-track changes to code we do not own.
3. **So the port became the cheaper route** to a requirement we had to meet anyway.

APPROACH

Agents plus a reference to check against



We ran **several Claude Code sessions in parallel**, each porting a different part of the code, instead of one engineer working through it in sequence.

The critical ingredient was **the reference output, not the model**. Without it, you have a lot of plausible code and no way to trust it.

VALIDATION

The suite found what the agents missed

An automated validation suite

Built in house by Trang Vo and Madhu Sridhar. Compares C and Rust outputs on **353 granules × 7 bands**, at the LaSRC stage and in the final HLS product, and logs runtime and memory in the same run.

Run across the full set, it surfaced bugs the first few test granules had hidden:

- Type conversions copied too faithfully
- Invalid auxiliary retrievals handled differently
- Scene-center geometry calculated, not read
- Bilinear instead of nearest-neighbor interpolation

Humans in the loop

Chris Holden worked through each class of issue with agent-driven refactoring and **manual review of every fix**. The agentic loop made this dramatically faster.

In two cases the model's choice was arguably **more correct** than the original. We matched the C code anyway, to prove equivalence first, and logged each difference to fix deliberately later.

AI accelerated the work, but it is **only as capable as the engineer or scientist using it**.

RESULTS

Same output, faster and cheaper

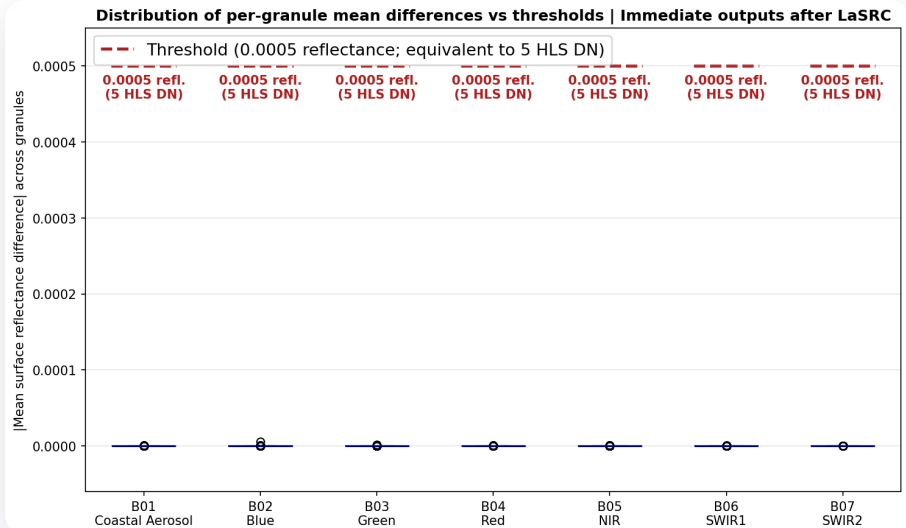
100%
of 353 granules within 5 DN on all 7 bands

30%
faster than the ESPA C code

7%
less memory

$21–36k
a year in EC2, based on 2025 EC2 spend of $240k

From targeted tweaks only. Structural performance work has not started.



Per-granule mean |difference| by band against the 5 DN threshold, 353 granules, Sept 2026 run

PART TWO

Next-generation infrastructure

Paying for processing, not for waiting

TODAY

What hurts in the current system

Step Functions
wakes on a timer, polls every job every 30 to 60 s

→

AWS Batch
one long job per granule: fmask, LaSRC, post-processing, upload

→

RDS log database
always on, pruned to stay fast

1 Granules silently dropped

Database timeouts before submission drop the job; timeouts afterwards leave finished work unrecorded and never retried.

2 Polling cost and rate limits

About 113 billed state transitions per granule, and AWS API limits at 10 to 20 thousand concurrent jobs.

3 Spot risk on long jobs

LaSRC is about half of Sentinel-2 runtime. An interruption late in a job throws away the most expensive work.

4 History pruned away

The audit trail needed for reprocessing and reconciliation is deleted to keep the database fast.

THE NEW DESIGN

Event-driven, with S3 as the log

Granule lands
S3 event, no timer

→

Lambda
records it, checks ancillary data

→

Batch on Spot
scales to zero

→

EventBridge
job finished event

→

Job monitor
reads the outcome

→

S3 log
append-only, never pruned

Success, cloudy, low sun: recorded as final states

Spot interruption: retry queue, resubmitted automatically

Unexpected failure: dead-letter queue, redriven after a fix

The core principle: **S3 is the event store, the log and the coordination mechanism.** No database, no polling. Nothing runs while nothing is arriving.

Prototype built with Claude as a coding assistant.

COST

Orchestration cost, line by line

Annual cost	Current (polling)	New (event-driven)
Step Functions	$15,518	not used
RDS database	$4,375	not used
Lambda	$205	~$2
S3 polling for ancillary data	~$60	~$0
S3 event store	not used	~$180
SQS and EventBridge	not used	~$15
Total	**~$20,160**	**~$200**

~100×

cheaper to orchestrate the same 15,000 granules a day.

The 2025 bill agrees: Step Functions $12.4k and RDS $8.3k.

WHERE THE MONEY WENT

Paying to ask "are you done yet?"



A 20 to 30 minute job is polled every 30 to 60 seconds. Every poll is a **wait, a Lambda call and a decision**, and each one is billed.

Polling is **about $15,518 a year** in Step Functions alone. An event costs nothing until something actually happens.

REPROCESSING

A full reprocessing campaign

Orchestration cost to reprocess the full archive, about **30M granules** (for example, HLS v3):



From **$55–85k** to **about $4.1k**, and **no rate-limit ceiling** on how fast we can go.

SEAN: CONFIRM

The cost analysis gives $4.1k in one table and $2.7k in another. Which is right?

OBSERVABILITY

Questions we can now answer

? Did every product reach LP DAAC?

Compare our output log against the LP DAAC catalog, by date.

? Did we try every granule ESA and USGS published?

Compare our input log against CMR, by date.

? How much is cloudy or low sun?

Recorded as their own outcomes, not lumped in with failures.

? What fails, how often, and does retrying help?

Every attempt and exit code, queryable with Athena.

The full history is kept **indefinitely, for about $180 a year**. No database to prune, no extra service to run.

ROLLOUT

Proven, then phased

Already proven

The HLS-VI historical backfill processed about **30M granules** on this same pattern.

The log never needed pruning, retries handled Spot interruptions cleanly, and weekly progress reports were a single query.

0. **Phase 0, shadow.** The new job monitor watches the current system read-only. Zero risk, real data from day one.
1. **Phase 1, event-driven.** Retire Step Functions polling and the database.
2. **Phase 2, split jobs.** fmask first, so cloudy granules exit early and Spot is safer.
3. **Phase 3, reconcile.** Use the log to find what is missing and drive reprocessing.

PART THREE

Applying this to other SNWG solutions

What we would look for in any production system, and how we can help

ACROSS SOLUTIONS

Patterns we see

WHAT WE SEE		WHAT HLS LEARNED
Whole granules copied to local disk, sometimes more than once	→	**Read only the bytes you need**, straight from cloud-optimized sources
Long single-step jobs forced onto on-demand instances	→	**Split jobs** so a Spot interruption costs minutes, not hours
Always-on servers and databases, sometimes two orchestrators at once	→	**Event-driven, scale to zero**: pay only when something arrives
Legacy Fortran or C wrapped in layers of Python	→	**Port with agents**, held to a validation suite
Homemade audit scripts to check what was produced	→	**An append-only log** you can query
Reprocessing with no cost estimate up front	→	**Model the cost** before you run the campaign

Drawn from public code and documentation across several solutions. Not specific to any one team.

SELF-ASSESSMENT

Questions to ask about your own system

- [ ] Do you know your **cost per granule**, and what a full reprocessing would cost?
- [ ] Are you paying for anything **while nothing is arriving**?
- [ ] Do you copy **whole files** when you only need some of the bytes?
- [ ] Would a **Spot interruption** waste an hour of work?
- [ ] If a granule went missing last month, could you **prove it with one query**?
- [ ] Could you change your science code tomorrow and **prove nothing broke**?

If you answered "no" or "not sure" to any of these, **we have been there**, and we would be glad to compare notes.

MULTIPLIER

HLS gains carry downstream

OPERA

DSWx-HLS surface water and DIST-HLS disturbance products are built from HLS.

FIRMS-2G

Burned area mapping uses the HLS vegetation indices.

HLS-LST

Land surface temperature builds on the HLS record.

When HLS gets **faster, cheaper and more reliable**, so does every product built on it. Lower HLS latency is lower latency for them.

Newer solutions, such as **GABAN** and **MWOW**, can build these patterns in **from the start** rather than retrofit them later.

TAKEAWAYS

What we'd like you to take away

1. **Measure first**
   Know your cost per granule. You cannot reduce what you have not measured.

2. **Pay for events, not for waiting**
   For HLS, orchestration fell from about $20,160 to about $200 a year, and reprocessing from $55–85k to about $4.1k.

3. **Legacy science code is now affordable to modernize**
   AI agents made a 5–6 month port practical, but only because a validation suite proved every result.

4. **Talk to us**
   We can share our designs, cost models and validation tooling.

   SEAN: FILL IN
   What exactly we offer (reviews, office hours, shared templates) and contact details.